

Impression navigateur 58 mm — Guide technique React + Vite

Projet de référence : receipt-printer

Approche : HTML/CSS + dialogue d'impression (`window.print`) via iframe isolée

Format cible : Roll Paper 58 × 297 mm (Epson TM, etc.)

Date : août 2026

1. Objectif

Implémenter, dans n'importe quel projet **React + Vite**, une fonctionnalité d'impression de ticket / reçu :

- aperçu à l'écran ;
- impression via le dialogue système du navigateur ;
- page forcée en **58 × 297 mm** ;
- mémorisation du libellé d'imprimante en **session** ;
- logo SVG + QR code imprimables.

Ce guide reprend la méthode utilisée dans ce dépôt, réutilisable ailleurs.

2. Prérequis

Élément	Détail
Stack	React 18+ / 19, Vite
Navigateur	Chrome ou Edge (recommandé)
Imprimante	Thermique 58 mm installée sous Windows (ex. Epson TM)
Contexte	http://localhost ou HTTPS (sécurisé)

Limite importante : une page web **ne peut pas** sélectionner programmatiquement le nom du papier driver (« Roll Paper 58 x 297 mm »). On force les **dimensions** via CSS `@page` . L'utilisateur choisit une fois le format dans le dialogue ; Chrome le réutilise ensuite souvent.

3. Architecture

```
src/
  components/
    Receipt.jsx      # Markup du ticket
    Receipt.css      # Styles écran (+ fallback Ctrl+P)
    StoreLogo.jsx    # Logo SVG inline
    ReceiptQr.jsx    # QR → data URL
  hooks/
    usePrinterSession.js # sessionStorage imprimante
  print/
    printReceipt.js   # Iframe isolée + window.print()
  data/
    sampleReceipt.js  # Données d'exemple
  App.jsx             # UI + bouton Imprimer
```

Flux d'impression :

1. L'utilisateur clique sur **Imprimer**.
2. `printReceiptElement` clone le DOM `.receipt` dans une **iframe** cachée.

3. L'iframe charge un HTML minimal + CSS `@page { size: 58mm 297mm } .`
4. On attend polices / images (QR).
5. `iframe.contentWindow.print()` ouvre le dialogue.
6. Au `afterprint`, on nettoie l'iframe et on peut ouvrir la modal « nom d'imprimante ».

Pourquoi une iframe ? Pour isoler le ticket du layout de l'app (`min-height: 100vh`, sidebars) qui sinon produit du blanc / A4 / fragments fantômes.

4. Étape 1 — Créer le projet (si besoin)

```
npm create vite@latest mon-projet -- --template react
cd mon-projet
npm install
```

Optionnel (QR) :

```
npm install qrcode
```

5. Étape 2 — Données du ticket

Fichier `src/data/sampleReceipt.js` :

```
export const sampleReceipt = {
  store: {
    name: 'Café du Port',
    address: '12 Quai Saint-Pierre',
    city: '17000 La Rochelle',
    phone: '05 46 00 00 00',
  },
  ticketNumber: 'A-00482',
  date: '06/08/2026',
  time: '10:42',
  cashier: 'Marie',
  items: [
    { name: 'Espresso', qty: 2, unitPrice: 1.8 },
    // ...
  ],
  taxRate: 0.1,
  footer: 'Merci de votre visite !',
  qrPayload: 'https://exemple.com/ticket/A-00482',
}

export function lineTotal(item) {
  return item.qty * item.unitPrice
}

export function receiptTotals(receipt = sampleReceipt) {
  const subtotal = receipt.items.reduce((s, i) => s + lineTotal(i), 0)
  const tax = subtotal * receipt.taxRate
  return { subtotal, tax, total: subtotal + tax }
}

export function formatMoney(value) {
```

```
return `${value.toFixed(2).replace('.', ',')} EUR`  
}
```

Adaptez les champs à votre métier (commande, facture, etc.).

6. Étape 3 — Composant Receipt (markup)

1. Créez `src/components/Receipt.jsx` avec la structure :
 - en-tête (logo + magasin) ;
 - méta (n° ticket, date) ;
 - lignes articles ;
 - totaux ;
 - pied (message + QR).
2. Racine obligatoire : `<article className="receipt">` — c'est ce nœud qui sera cloné pour l'impression.
3. Ajoutez `className="no-print"` sur toute UI à exclure (boutons, panneaux).

Exemple minimal :

```
export default function Receipt({ data }) {  
  return (  
    <article className="receipt">  
      <header className="receipt__header">...</header>  
      <ul className="receipt__items">...</ul>  
      <footer className="receipt__footer">...</footer>  
    </article>  
  )  
}
```

7. Étape 4 — Styles écran

Dans `Receipt.css` :

- largeur visuelle ~58 mm ;
- police monospace type thermique ;
- option `zoom: 1.5` pour grossir l'aperçu sans changer la largeur papier logique :

```
.receipt {  
  --receipt-scale: 1.5;  
  --receipt-width: 58mm;  
  width: calc(var(--receipt-width) / var(--receipt-scale));  
  zoom: var(--receipt-scale);  
  font-family: 'Courier New', Courier, monospace;  
  font-size: 9px;  
}
```

Important : ne pas compter sur `@media print` de l'app seule si vous utilisez l'iframe : les styles d'impression principaux vivent dans `printReceipt.js`.

8. Étape 5 — Module d'impression iframe

Fichier `src/print/printReceipt.js` — points clés :

8.1 Dimensions papier

```
const PAPER_WIDTH_MM = 58
const PAPER_HEIGHT_MM = 297 // = "Roll Paper 58 x 297 mm"
```

8.2 CSS embarqué

Inclure dans la string CSS :

```
@page {
  size: 58mm 297mm;
  margin: 0;
}
html, body, .receipt {
  width: 58mm !important;
  margin: 0 !important;
}
```

8.3 Fonction d'impression

```
export function printReceiptElement(target, { onAfterPrint } = {}) {
  const receiptEl = target?.classList?.contains('receipt')
    ? target
    : target?.querySelector?.('.receipt')
  if (!receiptEl) { onAfterPrint?.(); return }

  const iframe = document.createElement('iframe')
  iframe.style.cssText =
    `position:fixed;left:-10000px;top:0;width:58mm;height:297mm;border:0;opacity:0`
  document.body.appendChild(iframe)

  const doc = iframe.contentDocument
  const win = iframe.contentWindow
  doc.open()
  doc.write(`<!doctype html><html><head><meta charset="utf-8">
    <style>${RECEIPT_PRINT_CSS}</style></head>
    <body>${receiptEl.outerHTML}</body></html>`)
  doc.close()

  const finish = () => {
    win.removeEventListener('afterprint', finish)
    iframe.remove()
    onAfterPrint?.()
  }

  // Attendre images (QR data URL) puis :
  win.addEventListener('afterprint', finish)
  win.focus()
  win.print()
}
```

8.4 Images / QR

- Préférez un **logo SVG inline** (pas de fetch).
- Générez le QR en **data URL** (`qrcode.toDataURL`) pour qu'il survive au `outerHTML` dans l'iframe.
- Attendez `img.complete` / `onload` avant `print()` .

9. Étape 6 — Brancher dans React

```
import { useRef } from 'react'
import Receipt from './components/Receipt'
import { printReceiptElement } from './print/printReceipt'

export default function App() {
  const receiptRef = useRef(null)

  function handlePrint() {
    printReceiptElement(receiptRef.current)
  }

  return (
    <>
      <button type="button" onClick={handlePrint}>Imprimer</button>
      <div ref={receiptRef}>
        <Receipt />
      </div>
    </>
  )
}
```

Le `ref` peut être sur le wrapper ou directement sur `.receipt`.

10. Étape 7 — Session imprimante (optionnel mais utile)

Le navigateur ne révèle pas l'imprimante choisie. On mémorise un **libellé** saisi après la première impression.

`src/hooks/usePrinterSession.js` :

```
const STORAGE_KEY = 'receipt-printer.session'

export function usePrinterSession() {
  const [printer, setPrinterState] = useState(() => /* read sessionStorage */)

  const setPrinter = (printerName) => {
    const next = { printerName, configuredAt: new Date().toISOString() }
    sessionStorage.setItem(STORAGE_KEY, JSON.stringify(next))
    setPrinterState(next)
  }

  const clearPrinter = () => {
    sessionStorage.removeItem(STORAGE_KEY)
    setPrinterState(null)
  }

  return { printer, setPrinter, clearPrinter }
}
```

UX typique :

1. Première impression → `afterprint` → modal « Nom de l'imprimante ».
2. Affichage du nom en session.

3. Bouton **Changer d'imprimante** → `clearPrinter()` + nouvelle impression.

11. Étape 8 — Logo et QR

Logo

Composant SVG React (`StoreLogo.jsx`) avec `currentColor` — s'imprime en noir dans l'iframe.

QR

```
import QRCode from 'qrcode'
import { useEffect, useState } from 'react'

export default function ReceiptQr({ value }) {
  const [src, setSrc] = useState('')
  useEffect(() => {
    QRCode.toDataURL(value, { width: 140, margin: 1, errorCorrectionLevel: 'M' })
      .then(setSrc)
  }, [value])
  return src ? <img className="receipt_qr" src={src} alt="" /> : null
}
```

12. Étape 9 — Checklist navigateur / imprimante

1. Ouvrir l'app (`npm run dev`).
2. Cliquer **Imprimer**.
3. Choisir l'imprimante Epson TM (ou équivalent).
4. **Plus de paramètres** → format **Roll Paper 58 x 297 mm** (pas A4).
5. Échelle : **Taille réelle** / 100 %.
6. Valider. Les impressions suivantes réutilisent souvent ce choix.

Si le blanc reste sous le ticket : le format A4 est encore sélectionné — ce n'est pas un bug CSS de largeur, c'est le format papier du dialogue.

13. Étape 10 — Pièges à éviter

Piège	Conséquence	Remède
<code>window.print()</code> sur toute l'app	Page A4, blanc, UI visible	Iframe + clone <code>.receipt</code>
<code>visibility: hidden</code> sur le body	Espace réservé = page haute	<code>display: none</code> sur <code>.no-print</code>
zoom CSS à l'impression	Fragments / 2 ^e page fantôme	<code>zoom: 1</code> dans le CSS iframe
Iframe 0x0	Mesures / rendu faux	Iframe dimensionnée 58x297 mm
Image QR URL externe	Manque à l'impression	Data URL
Espérer sélectionner le papier par JS	Impossible en web standard	@page + choix manuel 1x

14. Variantes

- **Simple** : `window.print()` + `@media print` (OK pour A4 / PDF, moins fiable pour thermique).
- **Iframe (recommandé ici)** : isolation totale du ticket.

- **ESC/POS (Web Serial / WebUSB)** : pas de dialogue, bytes bruts — autre architecture (hors scope de ce guide).

15. Fichiers de référence dans ce dépôt

Fichier	Rôle
src/print/printReceipt.js	Cœur d'impression 58×297
src/hooks/usePrinterSession.js	Session imprimante
src/components/Receipt.jsx	Ticket
src/components/Receipt.css	Aperçu écran
src/components/StoreLogo.jsx	Logo
src/components/ReceiptQr.jsx	QR
src/App.jsx	Orchestration UI

16. Résumé express (copier-coller mental)

1. Marquer le ticket avec `.receipt`.
2. Copier `printReceipt.js` (iframe + `@page 58mm 297mm`).
3. Appeler `printReceiptElement(ref)` au clic.
4. (Optionnel) `sessionStorage` pour le nom d'imprimante.
5. Première fois : choisir **Roll Paper 58 x 297 mm** dans le dialogue.

Fin du guide.